

Trabalho de Organização de Computadores[†]

João Luís Almeida Santos

¹Universidade Federal da Fronteira Sul, Chapecó, Santa Catarina, Brasil |

1 | Introdução

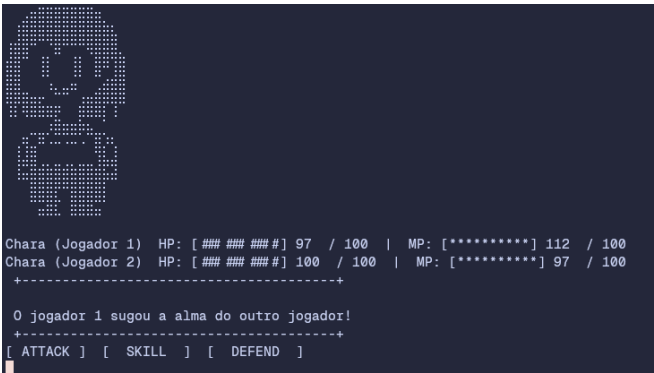


FIGURE 1 | Interface do simulador de batalhas no modo de observação.

O presente projeto documenta a tentativa de criar um simulador de batalhas de RPG, no estilo Pokémon (Figura 1), dentro do ambiente RISC-V, utilizando o RARS como ferramenta de simulação. A ISA utilizada é a RISC-V RV32IM, isto é, um conjunto de instruções que contempla apenas operações com inteiros, além de instruções de multiplicação e divisão.

O projeto, denominado *Undertale in Extremis*, foi desenvolvido como proposta alternativa às opções listadas na comanda, com

[†]Código fonte disponível em: https://github.com/lysz/undertale-in-extremis/protect_core

autorização do professor. O sistema simula batalhas entre dois bots autônomos que se enfrentam em turnos até que um deles seja eliminado. Cada bot recebe uma estratégia e age por conta própria, escolhendo a cada turno entre seis ações: ataque, defesa, Absolute Grit, Soul Suck, Final Execution e Mirror Shield. Ambos os bots iniciam com 100 pontos de vida (HP) e 100 de mana (MP), sendo o MP regenerado em +2 por turno.

TABLE 1 | Ações disponíveis por turno

Ação	Efeito	Custo
Ataque	1d20: erro <10, hit ≥10, duplica o dano em 20.	0 MP
Defesa	Bloqueia o próximo ataque. Contra-ataca quando crítico.	0 MP
Absolute Grit	Garante um crítico.	20 MP
Soul Suck	1–12 de auto-dano; ganha 4× em MP.	0 MP
Final Execution	800% de dano. Falha se alvo >50% HP.	150 MP
Mirror Shield	Reflete o próximo ataque recebido.	30 MP

O ataque realiza uma verificação 1d20: abaixo de 10 erra, acima acerta, e no 20 resulta em crítico com dano dobrado. A defesa bloqueia o próximo ataque recebido, podendo ripostar em um crítico.

A Absolute Grit garante um crítico ao custo de 20 MP. A Soul Suck causa 1–12 de dano ao próprio bot, drenando igual MP do inimigo e ganhando 4× esse valor, podendo ultrapassar o limite de 100 MP. A Final Execution causa 800% de dano ao custo de 150 MP, mas falha se o alvo estiver acima de 50% de HP. O Mirror Shield reflete o próximo ataque recebido de volta ao atacante, ao custo de 30 MP.

TABLE 2 | Atributos iniciais de cada bot

Atributo	Valor
HP	100
MP	100
Regeneração de MP	+2 por turno

TABLE 3 | Estratégias dos bots

Bot	Estratégia	Comportamento
Flowey	Aleatório	Escolhe uma das seis ações por RNG a cada turno, sem avaliar estado algum.
Chara	Inteligente	Acumula MP via Soul Suck até atingir 150 MP, reduz o inimigo abaixo de 50% de HP com Absolute Grit e então dispara a Final Execution.
Toby	Troll	Monitora o próprio HP e o MP do inimigo. Se estiver abaixo de 50 HP e o inimigo tiver ≥ 150 MP, levanta o Mirror Shield para refletir a Final Execution. Fora dessa janela, mistura ataques e Soul Suck, e executa o inimigo quando as condições permitem.

2 | Desafios e Soluções

2.1 | Randomização

Fazer um algoritmo que gerasse números realmente aleatórios foi desafiante. E para esse projeto, eu tive que misturar alguns métodos. A base principal foi o Xorshift, inventado por Marsaglia em 2003¹, publicado no Journal of Statistical Software. Em seu artigo, Marsaglia descreve:

A xorshift random number generator (xorshift RNG) produces a sequence of $2^{32} - 1$ integers x , or a sequence of $2^{64} - 1$ pairs x, y , or a sequence of $2^{96} - 1$ triples x, y, z , etc., by means of repeated use of a simple computer construction: exclusive-or (xor) a computer word with a shifted version of itself. In C, the basic operation is $y \wedge (y \ll a)$ for shifts left, $y \wedge (y \gg a)$ for shifts right. (In Fortran, a single form, `ieor(y, ishft(y, a))` will produce the desired result, with negative a for shifts right, nonnegative for shifts left.) Combining such xorshift operations for various shifts and arguments provides extremely fast and simple RNGs that seem to do well on tests of randomness. To give an idea of the power and effectiveness of xorshift operations, here is the essential part of a C procedure that, with only three xorshift operations per call, will provide $2^{128} - 1$ random 32-bit integers, given four random seeds x, y, z, w :¹

Em termos simples, o algoritmo pega um número aleatório, que é chamado de seed, faz três operações de XOR com versões deslocadas de si mesmo, e o resultado é o próximo número da sequência. A seed é atualizada a cada chamada. No entanto, havia um problema. A seed colocada no .data era sempre a mesma, então o algoritmo de randomização era na verdade, nada aleatório. Por isso, foi necessário utilizar a chamada de sistema de número 30 para multiplicar a seed pelos milissegundos do horário atual. No caso do número estourar, isso é tranquilo, pois o overflow simplesmente descarta os bits mais altos. O extttremu é usado por dois motivos para limitar o resultado ao range desejado via módulo e por ser a versão sem sinal da instrução, garantindo que o resultado nunca seja negativo, já que o xorshift pode produzir valores negativos.

```
randomizer:
    mv     s0, a0           # salva o range maximo
    la     t3, seed
    lw     t0, 0(t3)        # carrega a semente atual

    slli   t1, t0, 13
    xor    t0, t0, t1       # xorshift esquerda 13
    srli   t1, t0, 17
    xor    t0, t0, t1       # xorshift direita 17
    slli   t1, t0, 5
    xor    t0, t0, t1       # xorshift esquerda 5

    remu   s1, t0, s0       # limita ao range; remu evita
    ↪     negativos
```

```

addi    s1, s1, 1          # resultado em [1, range]

li      a7, 30

ecall                                # le o tempo atual do sistema

mul     t5, a0, t0

xor     t5, t5, t0          # mistura tempo com xorshift

sw      t5, 0(t3)           # atualiza a semente

mv      a1, s1

```

2.2 | Gerenciamento de Stack

O programa usa macros para abstrair e automatizar as operações necessárias de gerenciamento da stack durante chamadas de função. Essa estratégia facilitou o desenvolvimento e reduziu a ocorrência de erros relacionados ao gerenciamento da stack. O código apresentado corresponde a uma versão compatível apenas com o RARS, visto que, em outros *assemblers*, utiliza-se a diretiva `.endm` em vez de `.end_macro`. Aprendi essa técnica no último semestre de ORG.

```

.macro    startF
addi     sp, sp, -20
sw       ra, 16(sp)
sw       s0, 12(sp)
sw       s1, 8(sp)
sw       s2, 4(sp)
sw       s3, 0(sp)
.end_macro

.macro    endF
lw       ra, 16(sp)
lw       s0, 12(sp)
lw       s1, 8(sp)
lw       s2, 4(sp)
lw       s3, 0(sp)
addi     sp, sp, 20
.end_macro

```

Mesmo com os macros, o programa chegou a crashar durante o desenvolvimento. Algumas habilidades saíam da função via `j` para outro ponto do código em vez de retornar normalmente. Nesses casos, o `endF` nunca era executado: a stack crescia a cada uso da habilidade. A correção foi mover o `endF` para dentro de `do_attack_crit_no_message`, que passa a desalocar a stack de quem saltou para ele.

```

do_absolute_grit:
    startF
    call calculate_damage
    j do_attack_crit_no_message # endF nunca é chamado

# para consertar o problema foi necessario redefinir a funcao de
↳ critico, e nessa versao alternativa,
# há um endF

```

2.3 | Animação do modo de observação

Outro desafio foi fazer as animações de forma bonita e em tempo real. Para esta parte, foi usada a Gemini Pro 3.0 para gerar os frames além do Ascii original, obtido de². Isso pelo fato de que todo personagem tem 5 frames de animação, e estes são definidos dentro do `.data`. A forma de limpar a tela no terminal também foi algo que eu descobri pedindo para o Gemini Pro 3.0.

```

# quando o terminal l esse byte, ele d um espaamento do tamanho
↳ atual da tela em que voc est.

clear_screen: .byte 27, 91, 50, 74, 27, 91, 72, 0

# ento na verdade ele no limpa o terminal, ele s d um pulo pra
↳ baixo.

```

Para o modo de simulação, a animação era muito lenta, pois utilizada a instrução 32 várias vezes para esperar 500 - 1000 milissegundos para que as animações dessem a falsa impressão de serem fluídas. Sem esse wait, elas não podem ser vistas ou passam rápido demais. Por conta disso, o modo de benchmarking não tem animações, e portanto também não possui wait, e roda muito mais rápido. O modo é controlado por uma flag `modo_play` no `.data`: quando ativa, o programa pula todo o bloco de renderização e vai direto para a próxima ação.

```

la t0, modo_play
lw t0, 0(t0)
bnez t0, do_turn_action # em benchmark pula a animacao

```

Os bytes do `clear_screen` correspondem à sequência de escape ANSI `ESC[2J` seguida de `ESC[H`: o primeiro apaga o conteúdo visível e o segundo move o cursor para o canto superior esquerdo. O efeito visual é o de limpeza, mas o histórico do terminal continua acessível com scroll.

2.4 | Habilidades

O randomizador calculava o dano em `t1`, registrador temporário que não é preservado por `ecall`. No meio da função, um `ecall`

para leitura do tempo atualizava a semente do gerador, corrompendo o valor em `t1`. O dano resultante virava negativo, e a subtração de HP funcionava como cura.

```
remu    t1, t0, s0 # resultado corrompido pelo ecall seguinte
addi    t1, t1, 1
```

A correção foi mover o resultado para `s1`, que é preservado por chamadas de sistema.

Ao verificar o status do escudo, o código calculava o offset do array com `slli t2, t2, 4` em vez de `slli t2, t2, 2`. Cada elemento do array ocupa uma word (4 bytes), então o shift correto é de 2 bits. Com shift de 4, o offset era 4 vezes maior que o esperado, lendo uma posição de memória completamente errada e fazendo o escudo nunca ativar.

O primeiro problema era de lógica na decisão: sem um `j` explícito após a verificação de HP, o código caía diretamente na checagem de MP do inimigo independentemente da condição, fazendo o Toby levantar o escudo com frequência errada.

```
ble t5, t6, decision_troll_check_enemy_mp
# faltava: j decision_troll_not_execute
decision_troll_check_enemy_mp:
```

O segundo era de design: fora da janela do escudo, Toby usava Soul Suck como ação padrão. Isso causava auto-dano constante sem benefício real contra Flowey, que nunca acumula MP. A correção foi trocar Soul Suck por uma mistura de ataque e defesa. Houveram outros problemas documentados, só que alguns deles se mantiveram como features reais. A execução não foi feita para causar 800% de dano, foi pensada para apenas causar o dobro. Mas houve um erro de cálculo e uma junção de multiplicações, e, nesse caso, foi mantido o bug porque realmente era mais certo.

3 | Conclusão

O projeto resultou em um simulador de batalhas funcional em assembly RISC-V, com três bots e um modo de benchmark que roda 10.000 partidas automaticamente. Os resultados do benchmark foram contraintuitivos, conforme a Tabela 3. Normalmente, esperaria-se que os algoritmos mais planejados seriam mais eficazes, no entanto, o contrário ocorreu.

Bot	Taxa de vitória
Flowey	~52%
Toby	~29%
Chara	~17%

Flowey, o bot aleatório, ganhou a maioria das partidas. Chara, a mais elaborada, ficou em último. Com apenas seis ações e dano alto por turno, uma estratégia cuidadosa não tem vantagem real sobre o caos. O plano original incluía mais bots, mas balancear se mostrou difícil, e o espaço de ação era pequeno demais para que um quarto bot fizesse algo diferente dos três já existentes.

Na parte técnica, o projeto exigiu lidar com coisas que linguagens de alto nível escondem: stack manual, preservação de registradores e controle de fluxo via saltos. Os bugs foram difíceis de achar justamente por serem silenciosos, como o stack leak da Absolute Grit e o ataque que curava por corrupção de registrador.

4 | Anexos

References

- George Marsaglia. Xorshift RNGs. *Journal of Statistical Software*, 8(14):1–6, 2003.
- Undertale ASCII art. <https://emojicombos.com/undertale-ascii-art>. Acesso em: maio de 2026.